App Dev Project Report

1. Student Details

Name: Tushnamaitty P Nakra
Roll Number: 25F1002101
Email: 25f1002101@ds.study.iitm.ac.in
About Me: I am a student in the IIT Madras BS Degree Program in Data Science and Applications with a strong interest in software engineering, backend systems, and full-stack web development. I enjoy building real-world applications that combine scalable backend architectures, database systems, and interactive user interfaces.

---

2. Project Details

Project Title: Placement Portal

Problem Statement:

To design and develop a full-stack placement management system that enables students to create profiles, upload resumes, and apply for job drives, while allowing companies to create profiles, post drives, and manage applicants. The system also provides an admin interface to monitor overall activity, approve companies, and manage job postings. It simulates a real-world campus recruitment workflow by incorporating features such as application tracking, interview scheduling, and final selection processes.

Approach:

The application is built using:

- **Flask (Backend API)** for handling business logic and REST APIs
- **Vue.js (Frontend)** for dynamic UI and dashboards
- **SQLite + SQLAlchemy** for database management
- **JWT Authentication** for secure login and role-based access
- **Celery + Redis** for background jobs like:
  - Interview reminders
  - Placement report generation
- **Flask-Caching (Redis)** for optimizing frequently accessed APIs

---

3. AI/LLM Declaration

I used **ChatGPT (GPT-5)** to assist in writing SQLAlchemy model definitions, creating API documentation samples, and improving variable naming consistency.

The extent of AI/LLM usage is around **10–15%**, limited to **code suggestions and documentation formatting**.

All final implementation logic, debugging, and integration were done manually.

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Core backend web framework
SQLAlchemy	Object Relational Mapper for SQLite database
Vue.js	Frontend UI
Flask-JWT-Extended	Flask authentication
Redis	Cache + message broker
Flask-Caching	API caching
SQLite	Lightweight local database for storing user data
Celery	Background task Processing

---

5. Database Schema / ER Diagram

Tables:

Tables:
User

- id
- name
- email
- password
- role (admin / student / company)
- phone
- is_active
- is_approved
- created_at

StudentProfile

- id
- user_id
- department
- year
- cgpa
- skills
- resume
- placement_status

CompanyProfile

- id
- user_id
- company_name
- industry
- website
- location
- company_size

Job (Drive)

- id
- company_id
- title
- description
- skills
- salary
- location
- eligibility
- deadline
- is_approved
- is_close

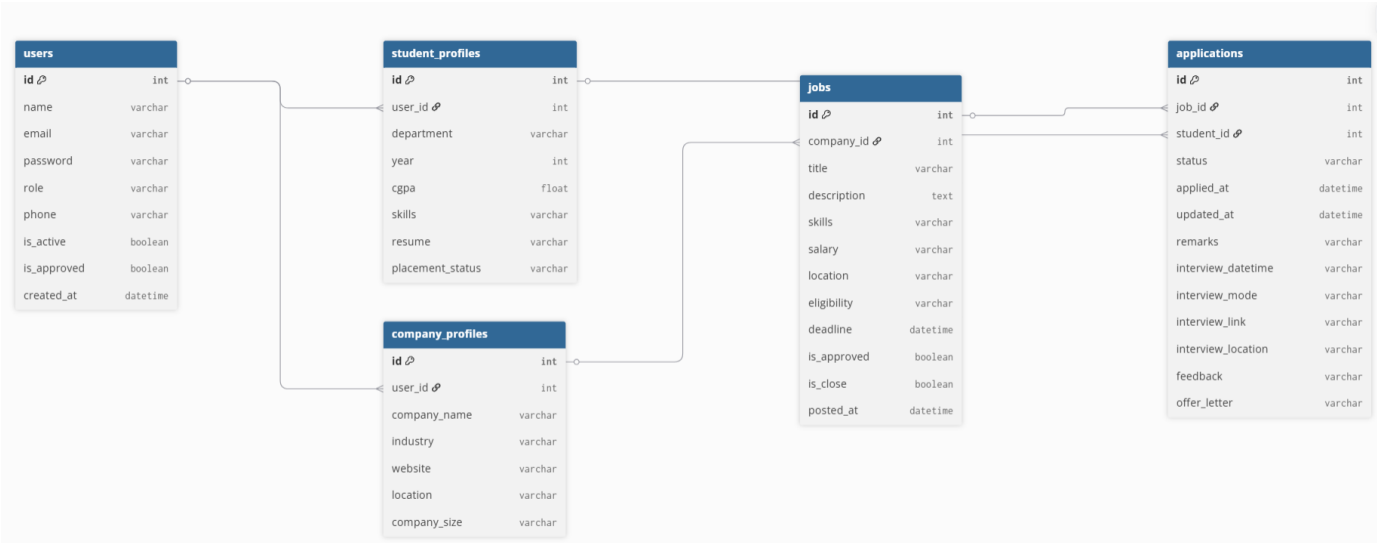
- posted_at

Application

- id
- job_id
- student_id
- status
- interview_datetime
- interview_mode
- interview_link
- feedback
- remarks
- updated_at

Relationships:

- **One-to-One:**
  - User → StudentProfile
  - User → CompanyProfile
- **One-to-Many:**
  - CompanyProfile → Job
  - StudentProfile → Application
  - Job → Application



---

6. API Resource Endpoints

Endpoint	Method	Description
Authentication APIs		
/register	POST	Register a new user (student/company/admin)
/login	POST	Authenticate user and return JWT token
Student APIs		
/student/profile	GET	Fetch student profile details
/student/profile	POST	Create or update student profile
/student/jobs	GET	Get all available job drives
/student/apply/{job_id}	POST	Apply for a job
/student/applications	GET	View all applications of student
/student/company/{id}	GET	View company details and jobs
Company APIs		
/company/create_profile	POST	Create company profile
/company/get_profile	GET	Fetch company profile
/company/create_drive	POST	Create a new job drive
/company/my_drives	GET	Get all drives posted by company
/company/open_drive/{id}	POST	Reopen a closed drive
/company/close_drive/{id}	POST	Close a drive
/company/update_drive/{id}	PUT	Update drive details
/company/drive/{id}/applications	GET	View applications for a drive
/company/application/{id}/update	POST	Update application status

Admin APIs		
/admin/dashboard	GET	Get platform statistics
/admin/companies	GET	View all companies
/admin/company/{id}/approve	POST	Approve a company
/admin/company/{id}/reject	POST	Reject a company
/admin/drives	GET	View all job drives
/admin/drive/{id}/approve	POST	Approve a drive
/admin/drive/{id}/reject	POST	Reject a drive
/admin/applications	GET	View all applications
/admin/search	GET	Search students/companies
/admin/user/{id}/toggle	POST	Activate/deactivate user
Reports & Background APIs		
/admin/generate-report	GET	Start report generation (Celery task)
/admin/report-status/{task_id}	GET	Check report generation status
/admin/download-report/{filename}	GET	Download generated report
/company/monthly-report	GET	Get company analytics report
/company/download-report	GET	Download latest company report

---

7. Architecture and Features (optional)

Architecture Overview:

- app.py : Main backend application (Flask + Celery setup)
- models : SQLAlchemy models (User, Job, Application, etc.)
- routes : API endpoints for student, company, admin
- frontend (Vue) : UI dashboards
- uploads/ :Resume storage
- reports/ : Generated CSV reports

Implemented Features:

- User authentication (JWT-based)
- Role-based access (Admin / Company / Student)
- Job drive creation & management
- Student job applications
- Application tracking (Applied / Shortlisted / Selected / Rejected)
- Interview scheduling
- Resume upload system
- Admin approval system
- Background jobs (Celery)
- Email notifications
- Report generation (CSV)
- API caching (Redis)

Additional Features:

- Interview reminder emails
- Placement analytics reports
- Export application data
- Company performance tracking

---

8. Video Presentation

Drive Link:
https://drive.google.com/file/d/1XQuj2y1_fVVqMZ5TU66fKuOhvmssNMPS/view?usp=sharing
(Accessible to all with "View" permission.)